[제안서] 요구사항 기반 Self-Service Cloud Platform

2026-10-09 · 이진서

CFP 4장 요건에 맞춘 제안서입니다. 트랙 D의 예시 주제 "10분 안에 production-ready 환경"을 출발점으로 삼았습니다. 수치는 실제 AWS(ap-northeast-2, 계정 1개)에서 측정한 값이며, 측정 전인 값은 "목표" 또는 "예상"으로 표시했습니다.

1. 제목 / 트랙

항목	내용
제목	앱 요구사항에 답하면 AWS 환경을 추천·생성·검증·회수하는 Self-Service Cloud Platform
트랙	D. Platform Engineering (실행 경로에 AI Agent 없음, 입력 보조 agent는 선택)
Project Lead	이진서
Reviewer	선수 2번, 3번

2. 문제 정의

AWS에 앱 하나를 올리려면 개발자가 compute 방식, 네트워크, IAM, 로그, 보안 헤더, 비용 구조를 모두 직접 정해야 합니다. 같은 "웹 서비스"라도 정적 사이트는 S3+CloudFront, 짧고 불규칙한 API는 Lambda, 연결을 오래 유지하는 서버는 Container가 맞는데, 이 판단을 사람마다 따로 하면 환경마다 구조와 보안 수준이 달라지고 쓰지 않는 고정비나 과도한 권한이 남습니다. 더 큰 문제는 "만들었다"와 "안전하게 쓸 수 있다"가 다르다는 점입니다. 이 플랫폼을 만드는 동안에도 Terraform이 로컬에서 모두 통과한 뒤 실제 AWS에서만 드러나는 권한 누락과 한도 초과로 배포가 7번 멈췄고, `terraform apply`가 성공한 환경이 보안 기준을 지킨다는 보장도 없었습니다. 개발자가 AWS 서비스 대신 앱의 요구사항에 답하면 플랫폼이 구성을 고르고, 승인 후 만들고, 결과를 자동으로 검증하고, 다 쓰면 남김없이 회수하게 하는 것이 이 프로젝트가 풀려는 문제입니다. 개발·테스트 환경을 많이 운영하는 회사도 같은 문제를 겪습니다.

3. 목표와 최종 지표

요구사항에 답한 개발자가 AWS 서비스를 하나도 고르지 않고 검증된 환경을 10분 안에 받되, 만들어지는 모든 환경이 같은 보안·비용 기준을 자동으로 만족하고, 삭제 후에는 아무것도 남지 않게 하는 것이 목표입니다.

정량 지표

지표	현재	목표	측정 방법
승인 → READY 시간 (DEV)	Static 3분 58초, Serverless 1분 43초 (실측). Container는 공유 인프라 최초 생성 포함 약 18분, 이후 환경 약 5분 (예상)	모든 Blueprint 10분 이내. Container의 공유 인프라 최초 생성만 20분 이내	환경 기록의 승인 시각과 READY 판정 시각 차이. Blueprint별 3회 측정
보안 기준 위반 차단	READY 환경 FAIL 0 (Static, Serverless 실측). 검사마다 로컬 위반 주입 테스트 있음	READY 환경 FAIL 0, 실제 AWS 위반 주입 10종 100% 차단	PLAN·STATE·HTTP 검사 결과와 Negative E2E (일부러 잘못된 설정을 넣고 READY가 나오지 않는지 확인)
삭제 후 남는 리소스	3개 Blueprint 모두 0 (실측, Container는 공유 인프라 21개까지 함께 삭제)	0	삭제 후 read-only 조회 (Terraform state, IAM role, ECR, log group, VPC origin, CloudTrail 생성 이벤트)

정성 지표

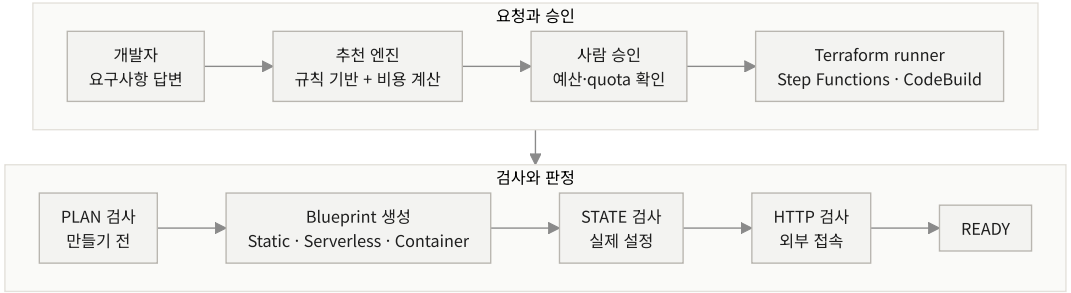
- 다른 선수 2명이 직접 환경을 만들어 보고 남긴 피드백: 질문이 이해하기 쉬운지, 실패했을 때 메시지만 보고 다음 행동을 정할 수 있는지
- 실제 AWS에서 겪은 장애마다 원인·수정·재발 방지 테스트가 남은 postmortem과 ADR

Scope

- **Must**: 요구사항 질문과 결정적(deterministic) 추천 엔진, DB 없는 Static·Serverless·Container DEV 환경의 생성·검증·삭제, 승인 전 비용 표시와 예산 상한, PLAN·STATE·HTTP 검증, 실패 원인 표시, Static 재배포·롤백
- **Nice-to-have**: Serverless + DynamoDB, Serverless 재배포·롤백, Container autoscaling, PROD 환경 E2E, 자연어 입력 보조 agent
- **Out of scope**: EKS, RDS, multi-region, blue/green 배포, custom domain, 여러 AWS 계정

4. 접근 방법

개발자는 AWS 서비스가 아니라 앱의 특징(트래픽, 가장 긴 요청 시간, 실시간 연결, 메모리 상태, 데이터, 멈춤 허용, 공개 범위)에 답합니다. 추천 엔진은 같은 입력에 항상 같은 결과를 내는 규칙 기반이고, 사람이 승인해야만 실제 AWS가 바뀝니다. READY는 `terraform apply` 성공이 아니라, 만들기 전 plan 검사, 만든 뒤 실제 설정 검사, 외부 접속 검사를 모두 통과했을 때만 줍니다.



세 검사 중 하나라도 기준을 어기면 READY가 되지 않고 FAILED와 실패 단계·원인이 기록됩니다. 보안 위반이면 접속도 바로 차단됩니다.

Blueprint	구조	이렇게 정한 이유
Static	비공개 S3 + CloudFront(OAC)	고정비 0, S3 직접 접근 차단
Serverless	사용자 container image + Lambda Web Adapter → Lambda → HTTP API	요청이 없으면 비용 0. 원본 image는 그대로 두고 adapter를 얹은 image를 플랫폼이 따로 만듦
Container	CloudFront → VPC origin → 공유 내부 ALB → Fargate(공인 IP, NAT 없음)	실시간 연결·긴 요청·메모리 상태가 필요한 앱용. ALB는 인터넷에 노출하지 않고 NAT Gateway 고정비를 피함

결정	선택	이유
추천 방식	규칙 기반 엔진 (LLM 사용 안 함)	같은 입력에 같은 결과, 추천 이유를 코드로 설명할 수 있음
실행	Step Functions + CodeBuild에서 Terraform, 환경별 state	고정비 0, 실행 이력이 남고 실패 단계가 분명함
권한	runner와 환경 role 모두 permissions boundary, `sscp-env-*` 이름으로 범위 제한	플랫폼이 강한 권한을 가져도 다른 리소스나 플랫폼 자신을 건드리지 못하게
공유 인프라	첫 Container 환경이 만들고 마지막 환경이 삭제	Container 환경이 없을 때 ALB 고정비 0
보안 위반 시	판정 즉시 접속 차단(CloudFront 비활성, Lambda 동시성 0)	위반한 환경이 잠깐이라도 열려 있지 않게

AI Agent 활용과 사전 학습 적용 계획
(필수 강좌 4개는 제안서 제출 전에 수강하고, 아래처럼 적용합니다)

- **역할**: 실행 경로에는 agent를 넣지 않습니다. 선택 기능인 입력 보조 agent는 "채팅 서버인데 사용자가 100명쯤"처럼 자연어로 쓴 설명을 질문 답변 초안으로 바꾸기만 하고, 사람이 확인한 답만 추천 엔진에 들어갑니다. 생성·삭제 권한은 없습니다.
- **AI Capabilities and Limitations**: LLM은 같은 질문에도 답이 달라질 수 있다는 전제로, 추천과 승인 판단은 규칙 기반 로직이 하고 agent는 입력 초안만 맡습니다. 강좌에서 정리한 "agent가 틀리는 지점"을 입력 보조 agent의 평가 항목으로 씁니다.
- **Claude Code 101**: repository에 project context 파일을 두고, AI coding tool에는 AWS read-only 조회만 맡깁니다. `terraform apply`와 삭제는 사람이 plan을 확인한 뒤 실행합니다.
- **Building Effective Human-Agent Teams**: "AI가 plan을 검토하고 사람이 승인한다"는 역할 분담으로 운영합니다. AI가 놓친 권한 누락을 사람이 실행 결과로 발견한 사례를 postmortem에 남깁니다.
- **Introduction to MCP**: Linear MCP를 연결해 실제 AWS에서 발견한 문제를 Issue로 만들고 engineering log와 연결합니다.
- **비교 방법**: 다른 선수 2명이 agent가 있을 때와 없을 때 요구사항 입력을 마치는 시간과 잘못 고른 답의 수를 비교합니다.

5. 예상 비용

플랫폼 자체는 사용량 과금 서비스로만 만들어 고정비가 거의 없습니다. 환경 비용은 플랫폼의 단가표(2026-10-04 기준 근사값)로 계산했고, 제출 전 AWS Pricing Calculator로 다시 확인합니다.

항목	월 예상 비용 (USD)
Control Plane (Cognito, API Gateway, Lambda, DynamoDB, Step Functions, S3, CloudFront, CloudTrail 1개)	약 1–2
CodeBuild runner (환경 생성·삭제할 때만 분 단위 과금)	1 미만
Static DEV 환경 1개	1 미만
Serverless DEV 환경 1개	1 미만
Container DEV 환경 1개 (Fargate Spot 1개 + 공유 ALB를 4개 환경이 나눔)	최대 12, 혼자 쓰면 약 23.5
실제 AWS E2E 테스트	별도 예산 10
합계 (테스트 제외, Container 1개 운영 기준)	약 15–27

비용 안전장치

- AWS Budgets 월 $30: 실제 비용 80%, 예측 비용 100%에서 메일 알림
- 플랫폼 전체 상한 $30: 새 환경의 최대 비용을 더하면 넘을 때 승인 단계에서 차단 (사용자가 우회할 수 없음)
- 사용자당 환경 수 제한 DEV 2개·PROD 1개, DEV 환경은 7일 뒤 자동 삭제
- NAT Gateway, 고정 IP(EIP), EC2 instance는 만들기 전 검사에서 차단
- 입력 보조 agent를 붙이면 CFP 8장의 네 가지 Bedrock 한도(저가 모델만 허용, 프로젝트 tag inference profile, 일일 token 알람, Budgets action)를 먼저 설정

6. 예상 리스크

가장 큰 리스크는 로컬 테스트를 다 통과해도 실제 AWS에서만 드러나는 문제입니다. 구현 중 이미 7번 겪었기 때문에, 새 Blueprint는 DEV 환경 하나로 실제 AWS에서 생성부터 삭제까지 확인한 뒤에만 완료로 봅니다.

리스크	영향	대응
실제 AWS에서만 드러나는 IAM 누락·한도 (실제: `apigateway:TagResource`, log delivery 권한, `ecs:ListServiceDeployments`, `ecs:ListTasks`, inline 정책 10,240자 한도, Linux용 provider checksum)	생성·검증 실패	실패마다 회귀 테스트로 고정. 검증 코드의 AWS 호출마다 필요한 권한을 공식 레퍼런스 기준 "권한 지도"로 테스트
검증기 오탐 (실제: 공유 route table을 위반으로 판정해 정상 환경을 차단)	정상 환경이 차단됨	테스트용 가짜 응답을 실제 AWS 응답 형태로 맞추고, E2E 응답을 테스트 데이터로 보관

리스크	영향	대응
Container 고정비 (ALB, 항상 실행되는 task)	방치하면 비용 누적	공유 ALB는 마지막 환경과 함께 삭제, 예산 상한, DEV TTL 7일
공유 인프라 생성·삭제가 오래 걸림 (VPC origin 생성 9분, 삭제 7분)	첫 Container 생성이 20분 가까이	진행 단계 메시지 표시, CodeBuild 90분 timeout, 두 번째 환경부터 공유 인프라 재사용
AI가 만든 코드·권한의 오류	잘못된 설정 배포	사람이 plan 확인 후 apply, 설명할 수 없는 코드는 merge하지 않음

7. 학습 계획

ECS Fargate와 CloudFront VPC origin, IAM 권한 레퍼런스는 이번에 처음 깊게 다루는 영역이라, Container 완료 전까지 아래 학습을 병행하고 engineering log에 `학습` tag로 남깁니다.

기술·서비스	현재 수준	필요한 이유	공부 방법
ECS Fargate, ALB, CloudFront VPC origin	첫 E2E에서 생성까지 성공	Container Blueprint 완성	공식 문서, 실제 E2E 응답 분석, 두 번째 환경으로 공유 인프라 재사용 확인
IAM Service Authorization Reference	실패를 통해 읽기 시작	action마다 필요한 resource type이 달라 누락이 반복됨	검증 코드의 AWS 호출별 권한 지도 작성, IAM Access Analyzer policy 검증
Terraform provider·state·lock	실제 장애 경험 있음	Windows 개발과 Linux CodeBuild 실행 환경 차이	여러 플랫폼용 lock 파일, saved plan과 state serial 동작 실습
AWS 비용 모델	근사 단가표로 계산	추천 순위와 예산 상한의 정확도	Price List API로 단가 갱신, 실제 청구서와 비교
Claude Academy 필수 강좌 4개	제안서 제출 전 수강 예정	CFP 필수, 4장의 agent 적용 계획 확정	Claude Code 101, AI Capabilities and Limitations, Introduction to MCP, Building Effective Human-Agent Teams (약 7시간)
Amazon Bedrock, MCP	처음	입력 보조 agent와 Linear 연결	Claude with Amazon Bedrock 중 tool use 부분, MCP 강좌 실습 반복

AI 사용 내역

항목	AI를 쓴 방식	받아들인 것 / 버린 것과 이유
2. 문제 정의	GPT와 1차 초안을 쓴 뒤, Claude에게 실측 기록을 기준으로 다시 검토를 받음	"Terraform을 대신 실행해 주는 도구와 무엇이 다르냐"는 관점을 받아들여, apply가 성공한 뒤에도 실제 AWS에서만 드러난 실패 7건을 근거로 "만들었다 ≠ 안전하게 쓸 수 있다"를 문제의 중심에 둠. 확인되지 않은 수치는 버림
4. 접근 방법	공유 ALB와 환경별 ALB, NAT와 공인 IP 등 대안 비교를 받음	비용 기준으로 공유 ALB + NAT 없는 구조를 선택. 결정은 대안을 질문으로 비교해 직접 확정
IAM·Terraform	AI가 정책과 Terraform 초안을 작성	실제 AWS에서 6건이 누락돼 있었음. AI가 "불필요하다"며 지운 권한이 실제로 필요했던 사례가 있어, 권한을 줄일 때는 공식 레퍼런스 근거를 먼저 남기는 규칙을 만듦
검증 코드	AI가 검사 로직과 테스트를 작성	가짜 응답이 실제 AWS 응답과 달라 오탐이 났음. 실제 응답 형태로 테스트를 고침
5. 예상 비용	비용 표 초안 작성	항목 구성은 사용, 금액은 플랫폼 단가표와 Pricing Calculator로 직접 다시 계산

참고: 초기 Linear 구성

제안서 필수 항목은 아니지만, 3장의 성공 기준을 아래처럼 Milestone과 Issue로 옮겨 관리합니다.

Milestone	대표 Issue
1. Control Plane + Static	요구사항 질문, 추천 엔진, 승인 흐름, Static E2E (생성 → 재배포 → 롤백 → 삭제)
2. Serverless	Lambda Web Adapter image 생성, HTTP API, Serverless E2E
3. Container	공유 인프라, ECS Fargate, Container E2E, 두 번째 환경 재사용 확인
4. Hardening	Negative E2E 10종, 검증기 오탐 방지, 로그 버킷 versioning
5. 마무리	postmortem, engineering log, 최종 보고서, 15분 발표, 3분 demo

Issue는 team key를 붙여 `SSCP-12`처럼 번호가 매겨지고, 같은 ID를 branch 이름에 넣어 Pull Request와 연결합니다.